

Lab4 Conditional VAE for Video Prediction Report

Deep Learning, NYCU

朱驛庭, 111550093

1. Introduction

In this lab we implement a **conditional variational autoencoder (cVAE)** for pose-conditioned video prediction. Given a single past frame x_1 and a sequence of pose images $\{P_2, \dots, P_T\}$, the model autoregressively generates the remaining $T-1$ frames. The architecture follows the standard cVAE protocol (Fig. 1) and draws on two standard references: *Everybody Dance Now* [1] for the pose-conditioned generation idea, and *Stochastic Video Generation with a Learned Prior* [2] for the VAE-based latent-variable formulation.

The key training challenge is the well-known **KL-vanishing** problem of VAEs combined with **exposure bias** of autoregressive sequence models. We address both with: (i) cyclical / monotonic KL annealing [3]; (ii) a teacher-forcing schedule that gradually transitions the decoder from ground-truth conditioning to its own predictions; and (iii) a numerically stable KL criterion with logvar clamping and proper element-wise normalization. Our final model (Monotonic annealing, fixed pipeline) reaches a per-frame validation **PSNR of 38.87 dB** averaged over the 629 predicted frames. Our extended-training submission (Cyclical, 140 epochs) achieved **PSNR 35.84164 on the Kaggle public leaderboard, ranked 19th**.

The report is organized as follows: Sec. 2 describes the model, the training/testing protocol, and our concrete choices for the reparameterization trick, teacher forcing, and KL annealing. Sec. 3 compares the three KL annealing strategies, dissects the teacher-forcing schedule, presents the per-frame PSNR analysis, and discusses the bug-fix that took us from a collapsed PSNR $\approx$ 19 dB run to PSNR $\approx$ 38 dB.

2. Implementation Details

2.1. Model Overview

The five modules in `src/modules/` are kept structurally unchanged per the spec. They are wired together in `Trainer.py` as illustrated in Fig. 1. Let

$F = 128, L = 32, N = 12, D_{out} = 192$ denote the feature dimensions of the frame encoder, label encoder, latent, and decoder fusion output respectively, and $H \times W = 32 \times 64$ the resized frame size.

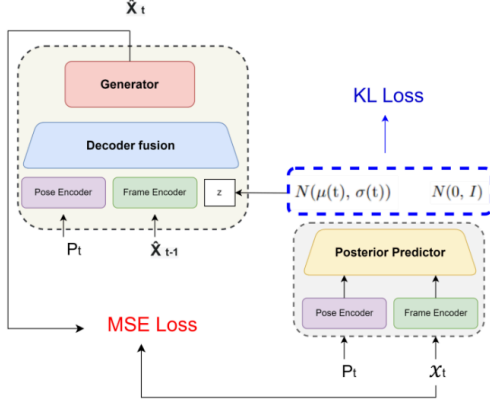
- **Frame Encoder** (`RGB_Encoder`): RGB frame $\rightarrow F$ -channel feature.
- **Pose Encoder** (`Label_Encoder`): pose image $\rightarrow L$ -channel feature.
- **Posterior Predictor** (`Gaussian_Predictor`): consumes the encoded current frame and current pose and outputs $(\mu, \log \sigma^2)$ of dimension N at the spatial size of the encoder.
- **Decoder Fusion** (`Decoder_Fusion`): concatenates the encoded *previous* frame, the encoded current pose, and the sampled latent z , then projects to D_{out} channels.
- **Generator**: D_{out} -channel feature $\rightarrow$ RGB image.

2.2. Training / Testing Protocol

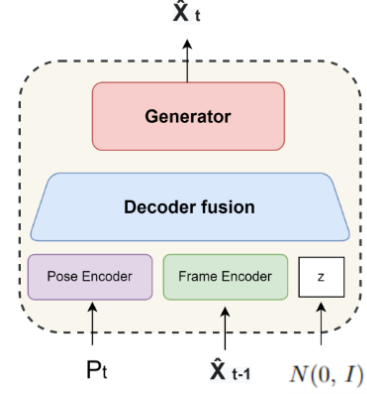
Training. Each training clip contains $T=16$ frames and 16 paired pose maps. We unroll the model frame-by-frame; at each step $t \in \{1, \dots, T-1\}$ the network sees the encoded previous frame, the encoded next pose P_{t+1} , and a latent $z_t \sim \mathcal{N}(\mu_t, \sigma_t^2)$ sampled by the *posterior* predictor (which gets to look at the ground-truth next frame x_{t+1}). The previous frame fed to the decoder is either the ground-truth x_t (*teacher forcing*) or the model's own previous output $\hat{x}_t$, decided by a Bernoulli draw with probability ρ_{tf} (per-batch). The per-clip loss is the time-summed VAE objective

$$\mathcal{L} = \sum_{t=1}^{T-1} \text{MSE}(\hat{x}_{t+1}, x_{t+1}) + \beta_{epoch} \sum_{t=1}^{T-1} \text{KL}(q_\phi(z_t | x_{t+1}, P_{t+1}) \| \mathcal{N}(0, I)).$$

We deliberately *sum* (rather than average) over time, following the cVAE-video recipe; this keeps the relative



(a) **Training.** Posterior Predictor takes the ground-truth current frame x_t and pose P_t and outputs $\mathcal{N}(\mu_t, \sigma_t^2)$, regularized toward $\mathcal{N}(0, I)$ via the KL loss. Sampled z_t together with the encoded previous frame $\hat{x}_{t-1}$ and pose P_t are fused and decoded into $\hat{x}_t$, supervised by MSE against x_t .



(b) **Inference.** The posterior is dropped: $z \sim \mathcal{N}(0, I)$ is sampled from the prior. Only $\hat{x}_{t-1}$ and P_t condition the prediction, fed back autoregressively for the full 629-frame rollout.

Figure 1. **Conditional VAE pipeline.** (a) Training uses the posterior network on (x_t, P_t) to produce a tight latent distribution; reconstruction is supervised by MSE and regularized by KL. (b) At inference time, the posterior is unavailable so z is sampled from the prior; predictions feed back autoregressively. Diagrams reproduced from the lab specification.

MSE-KL scale comparable to the senior reference and yields stronger gradient signal on long unrollings.

The full training loop:

```

1 def training_one_step(self, img, label, tf):
2     img = img.permute(1, 0, 2, 3, 4) #
3     label = label.permute(1, 0, 2, 3, 4)
4     T, B = img.shape[0], img.shape[1]
5
6     beta = self.kl_annealing.get_beta()
7     total_mse = total_kld = 0.0
8     last_pred = img[0]
9
10    for t in range(1, T):
11        prev = img[t-1] if tf else last_pred
12        enc_prev =
13            self.frame_transformation(prev)
14        enc_curr =
15            self.frame_transformation(img[t])
16        enc_pose =
17            self.label_transformation(label[t])
18
19        z, mu, logvar = self.Gaussian_Predictor(
20            enc_curr, enc_pose)
21        fused = self.Decoder_Fusion(
22            enc_prev, enc_pose, z)
23        pred = self.Generator(fused)
24
25        total_mse += self.mse_criterion(
26            pred.clamp(0,1), img[t])
27        total_kld += kl_criterion(mu, logvar, B)
28        last_pred = pred.detach().clamp(0,1)
29
30    loss = total_mse + (beta * total_kld
31                      if beta > 0 else 0.0)
32    self.optim.zero_grad()

```

```

30 loss.backward()
31 self.optimizer_step()
32 return loss, total_mse, total_kld

```

Two non-obvious details deserve emphasis:

1. **No NaN-mask in training.** A common template trick is to overwrite NaN predictions with a constant (e.g. 0.5) before computing the loss. We removed this because NaN→constant has *zero gradient*, creating an absorbing attractor: any clip that produces a NaN is replaced with a gray frame, the loss looks fine, and the model gradually learns to output gray (PSNR=15.28 dB constant). We rely on `logvar` clamping plus gradient clipping for numerical stability instead.
2. **detach() on the autoregressive feedback.** Otherwise BPTT spans the entire $T-1$ rollout, which explodes the moment teacher forcing turns off.

Testing / Inference. At test time there is no posterior — the latent is drawn from the prior $\mathcal{N}(0, I)$. To suppress per-frame noise that compounds over the 629-frame rollout we use the *prior mean* $z = \mathbf{0}$ (equivalent to noise scale 0). Validation is purely autoregressive: $\hat{x}_{t+1} = G(x_1, P_{2:t+1}, z = \mathbf{0})$ with $\hat{x}_t$ feeding back as the next “previous frame”.

```

1 @torch.no_grad()
2 def val_one_step(self, img, label):
3     img = img.permute(1, 0, 2, 3, 4)
4     label = label.permute(1, 0, 2, 3, 4)
5     T, B = img.shape[0], img.shape[1]
6     total_mse, psnrns = 0.0, []
7     prev = img[0]
8     for t in range(1, T):
9         enc_prev =
10             ↪ self.frame_transformation(prev)
11         enc_pose =
12             ↪ self.label_transformation(label[t])
13         z = torch.zeros(B, self.args.N_dim,
14             enc_prev.shape[-2],
15             ↪ enc_prev.shape[-1],
16             device=img.device)
17         fused = self.Decoder_Fusion(
18             enc_prev, enc_pose, z)
19         pred = self.Generator(fused).clamp(0,
20             ↪ 1)
21         pred = torch.nan_to_num(pred, nan=0.5)
22         total_mse += self.mse_criterion(pred,
23             ↪ img[t])
24         psnrns.append(float(
25             Generate_PSNR(pred, img[t])))
26     prev = pred
27     return total_mse / (T-1),
28         ↪ float(np.mean(psnrns))

```

2.3. Reparameterization Trick

The posterior predictor outputs $(\mu, \log \sigma^2)$ of shape (B, N, H', W') where $H' \times W' = 32 \times 64$ is the spatial size at the latent. We clamp $\log \sigma^2 \in [-5, 5]$ before exponentiation – without this, $\log \sigma^2 \approx 10$ gives $\sigma^2 \approx 22,000$, which produces Inf in the autoregressive rollout. The latent is then sampled as

$$z = \mu + \exp(\frac{1}{2} \log \sigma^2) \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

This keeps the sampling differentiable w.r.t. $(\mu, \log \sigma^2)$ while pushing the stochasticity into the parameter-free ϵ . The (per-element-normalized, numerically clamped) KL criterion is implemented as:

```

1 def kl_criterion(mu, logvar, batch_size):
2     logvar = torch.clamp(logvar, -5.0, 5.0)
3     KLD = -0.5 * torch.sum(
4         1 + logvar - mu.pow(2) - logvar.exp())
5     total = (batch_size * mu.shape[1]
6         * mu.shape[2] * mu.shape[3])
7     return KLD / max(1, total)

```

Why divide by all dimensions, not just batch?

The latent has $N \cdot H' \cdot W' = 12 \cdot 32 \cdot 64 = 24,576$ elements per sample. The standard “per-batch” KL would be $\sim 24k \times$ larger than the MSE loss, so any $\beta > 4 \cdot 10^{-5}$ would crush the reconstruction term. Normalizing by all dimensions makes KL and MSE comparable in magnitude so $\beta \in [0, 1]$ behaves intuitively.

2.4. Teacher Forcing Strategy

We use the standard scheduled-decay strategy: ρ_{tf} starts at 1.0, stays at 1.0 until `tfr_sde`, then linearly decays by `tfr_d_step` per epoch until reaching 0.0. The Bernoulli draw is *per training batch*, not per frame inside a clip — a clip is fully teacher-forced or fully autoregressive, so the gradient signal stays consistent within a single backward pass.

```

1 def teacher_forcing_ratio_update(self):
2     if self.current_epoch >= self.tfr_sde:
3         self.tfr = max(0.0, self.tfr -
4             ↪ self.tfr_d_step)
5
6 # in the training loop, once per batch:
7 adapt_TeacherForcing = (random.random() <
8     ↪ self.tfr)

```

For all three annealing strategies in our final comparison, ρ_{tf} starts at 1.0. For Monotonic and None we used `tfr_sde=10` (decay starts at epoch 10, finishes by epoch 19). For Cyclical we deliberately delayed decay by 5 epochs (`tfr_sde=14`, finishes by epoch 24) so that the model could establish a stable reconstruction during the first β -cycle before being asked to handle its own predictions; we discuss the consequences in Sec. 3. `tfr_d_step=0.1` throughout.

2.5. KL Annealing Schedule

We implement all three schedules in a single class and pick between them via `--kl_anneal_type`.

```

1 class kl_annealing():
2     def __init__(self, args, current_epoch=0):
3         self.type = args.kl_anneal_type
4         self.n_epoch = args.num_epoch
5         self.n_cycle = max(1,
6             ↪ args.kl_anneal_cycle)
7         self.ratio = args.kl_anneal_ratio
8         self.i = current_epoch
9         if self.type == 'Cyclical':
10             self.L = self.frange_cycle_linear(
11                 self.n_epoch, 0.0, 1.0,
12                 self.n_cycle, self.ratio)
13         elif self.type == 'Monotonic':
14             self.L = self.frange_cycle_linear(
15                 self.n_epoch, 0.0, 1.0,
16                 1, self.ratio)
17         else:
18             # 'None'
19             self.L = np.ones(self.n_epoch,
20                 dtype=np.float32)
21
22     def frange_cycle_linear(self, n_iter, start,
23         stop, n_cycle,
24         ↪ ratio):
25         L = np.ones(n_iter, np.float32) * stop
26         period = n_iter / n_cycle
27         step = (stop - start) / max(1.0,

```

```

25         period *
           ↳ ratio)
26     for c in range(n_cycle):
27         v, j = start, 0
28         while v <= stop and \
29             int(c*period+j) < n_iter:
30             L[int(c*period+j)] = v
31             v += step; j += 1
32     return L

```

The defaults (matching senior jayin92’s PSNR-37 recipe) are: `kl_anneal_cycle=10`, `kl_anneal_ratio=0.5` (proportion of each cycle spent ramping $0 \rightarrow 1$, the rest held at 1). For *Monotonic*, “cycles=1” so the schedule is one ramp from $0 \rightarrow 1$ over the first 50% of training, then held at 1. For *None*, β is constant at 1.

Figure 2 visualizes the three resulting β schedules for our 70-epoch run.

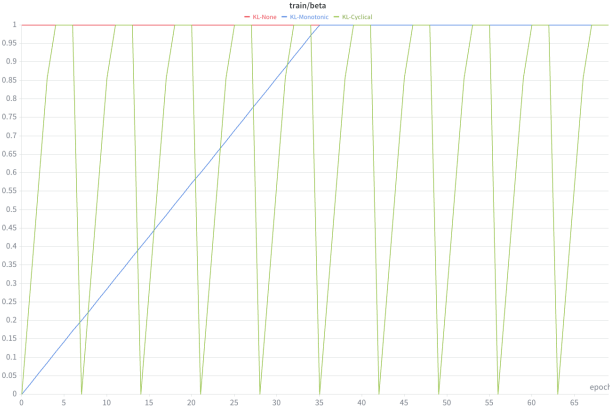


Figure 2. β schedule for each KL annealing strategy. *KL-None* (red) is constant at 1; *KL-Monotonic* (blue) ramps linearly from 0 to 1 over the first ~ 35 epochs then holds; *KL-Cyclical* (green) repeats a sawtooth with ~ 10 cycles of period ~ 7 epochs.

3. Analysis & Discussion

All three runs share the same model architecture, optimizer (Adam, $\text{lr}=10^{-3}$, CosineAnnealingLR with `eta_min=0`), batch size 2, $T_{\text{train}}=16$, $T_{\text{val}}=630$, and 70 epochs, after the bug fixes described in Sec. 3.5. The only differences are the KL schedule and the teacher-forcing decay start (delayed by 5 epochs for Cyclical, see Sec. 3.1).

3.1. Teacher Forcing Ratio

Figure 3 shows the teacher-forcing schedule actually used in the post-fix runs. Reading this together with the loss curves in Fig. 6 and Fig. 4 reveals two regimes:

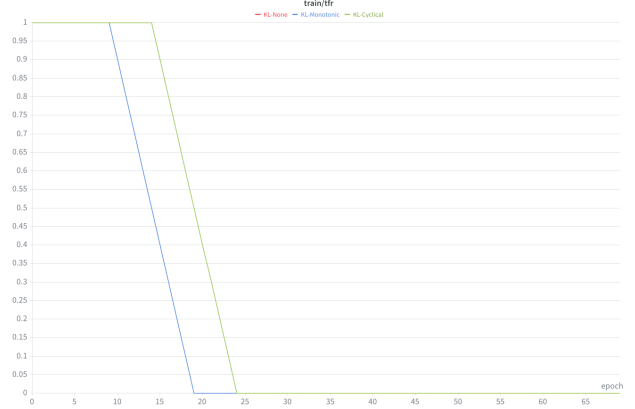


Figure 3. Teacher forcing ratio ρ_{tf} over training (post-fix runs). Monotonic and None use `tfr_sde=10` (decay completes at epoch 19); Cyclical is offset to `tfr_sde=14` (completes at epoch 24).



Figure 4. Validation PSNR (post-fix). Peak values: *KL-Monotonic* ≈ 38.5 dB, *KL-None* ≈ 38.4 dB, *KL-Cyclical* ≈ 36.8 dB. Note the temporary plateau during epochs 10–20 corresponding to the TFR-decay window.

- **Epochs 0–10 ($\rho_{tf}=1.0$):** the model trains essentially as a one-step conditional VAE. Validation PSNR climbs from ~ 15 dB to ~ 33 dB for all three runs almost identically. This is the regime where the model learns to reconstruct given a clean previous frame.
- **Epochs 10–20 (decay):** this is the *exposure-bias adaptation* window. As ρ_{tf} drops, the model is increasingly fed its own outputs and must learn to absorb its own errors. We see a clear bump in MSE (Fig. 5) around epochs 10–17 for *KL-None* and *KL-Monotonic* and a delayed-but-larger bump around epochs 15–19 for *KL-Cyclical* (because its decay starts 5 epochs later). PSNR *tem-*

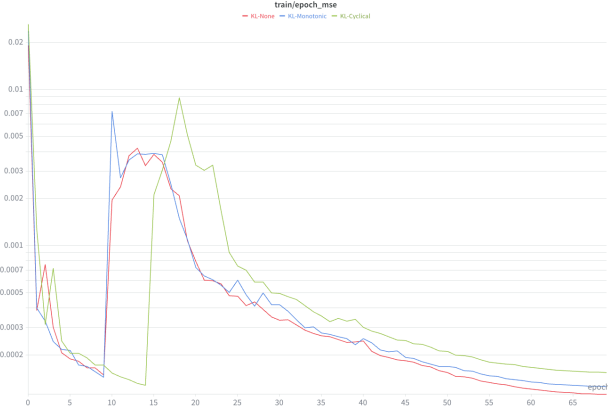


Figure 5. **Training MSE (post-fix, log scale).** Bump near epochs 10–17 (None, Mono) and 15–19 (Cyclical) is the TFR-decay transient, delayed by 5 epochs for Cyclical. Post-decay, KL-Cyclical (green) sits $\sim 50\%$ above the others.



Figure 6. **Validation loss (post-fix, log scale).** Final values: KL-None $\approx 1.6 \times 10^{-4}$, KL-Monotonic $\approx 1.7 \times 10^{-4}$, KL-Cyclical $\approx 2.5 \times 10^{-4}$.

porarily plateaus during this window and then resumes climbing, exactly as expected: the model briefly trades single-step reconstruction quality for autoregressive robustness.

- **Epochs 20–70 ($\rho_{tf}=0$):** pure autoregressive training. PSNR continues to climb to its final value, with the curves now stratified by KL strategy.

Why Cyclical’s TFR is delayed. An early experiment in which Cyclical used the same `tfr_sde=10` as the others showed Cyclical’s first β -reset (around epoch 7) coinciding with the start of TFR decay (epoch 10), causing a compound disturbance: the latent regularizer

suddenly relaxes and the model is simultaneously asked to consume its own predictions. We delayed Cyclical’s TFR decay to epoch 14 to give the model one full β -cycle of stable reconstruction first. As we show next, this still wasn’t enough — Cyclical’s repeated β shocks remain its main weakness.

3.2. KL Annealing Comparison

To isolate the effect of the KL schedule from confounding factors, this subsection compares the three strategies using the **pre-bug-fix runs** (`assets/prev/`). These three runs share an *identical* teacher-forcing schedule (Fig. 7), so any difference between them is attributable to the KL strategy alone — unlike the post-fix runs in Sec. 3.1 where Cyclical’s TFR was deliberately offset. The post-fix runs (which reach PSNR 36.8–38.5 dB) are discussed separately in Sec. 3.5; the absolute PSNR levels here (~ 19 dB, all collapsed to the constant-gray attractor described in Sec. 3.5) are not the point of this comparison — the *dynamics* are. All loss panels use a *logarithmic* y-axis.

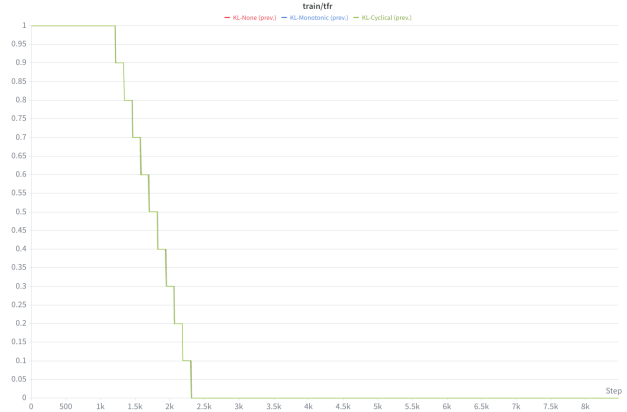


Figure 7. **Teacher forcing ratio (pre-fix runs).** All three KL schedules share an identical ρ_{tf} trajectory, making this a controlled comparison of the KL strategy alone.

What the dynamics reveal.

- **Cyclical induces structured perturbations.** The KLD curve (Fig. 9) is the cleanest evidence: each β -reset re-injects free entropy into the latent, which then gets squeezed back as β ramps up. The MSE curve (Fig. 10) shows the cost: every β -reset is followed by a reconstruction-quality dip as the latent posterior is forcibly relaxed. Monotonic and None show none of this oscillation.
- **Cyclical has the lowest equilibrium KLD.** Because every cycle re-pulls the posterior toward

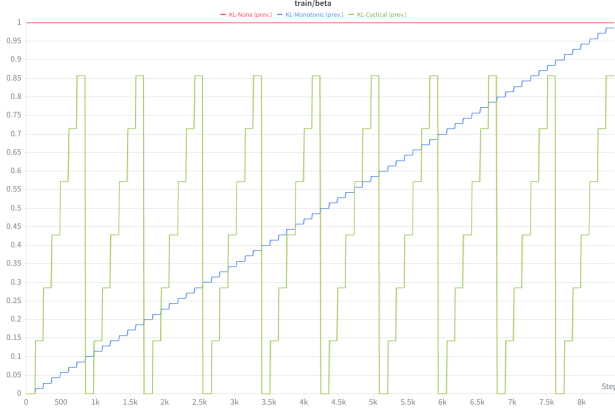


Figure 8. β schedule (pre-fix runs). *None* (red) constant at 1; *Monotonic* (blue) linear ramp $0 \rightarrow 1$ over the first half of training; *Cyclical* (green) sawtooth with period ~ 7 epochs.

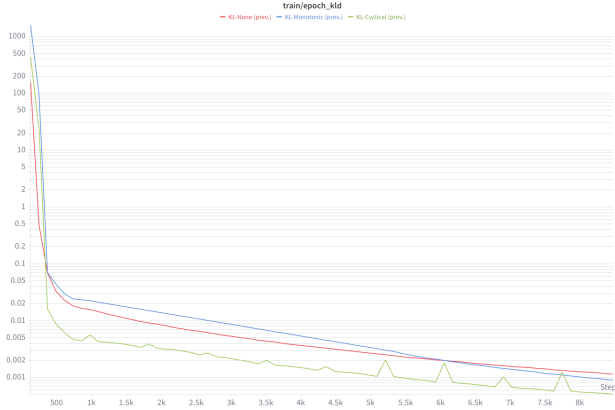


Figure 9. **Training KLD (pre-fix, log scale).** Cyclical (green) shows the characteristic periodic spike pattern aligned with each β -reset, while Monotonic and None are smooth. Final ranking: None ($\sim 1.1 \times 10^{-3}$) > Monotonic ($\sim 0.9 \times 10^{-3}$) > Cyclical ($\sim 0.5 \times 10^{-3}$).

$\mathcal{N}(0, I)$, Cyclical’s mean KLD is half that of None or Monotonic. In a normal regime this would be a feature – it would prevent posterior collapse. In our regime (large 24,576-dim latent, no collapse risk) it is a bug, because the KL term in our setup is *already* negligibly small ($\sim 10^{-3}$ here, $\sim 10^{-7}$ post-fix), so further compression only sacrifices reconstruction.

- **None has the lowest MSE.** A constant $\beta = 1$ provides a stable optimization landscape; the encoder/decoder are not periodically pulled in inconsistent directions. This is the hint that, in our setting, KL annealing is not buying us anything

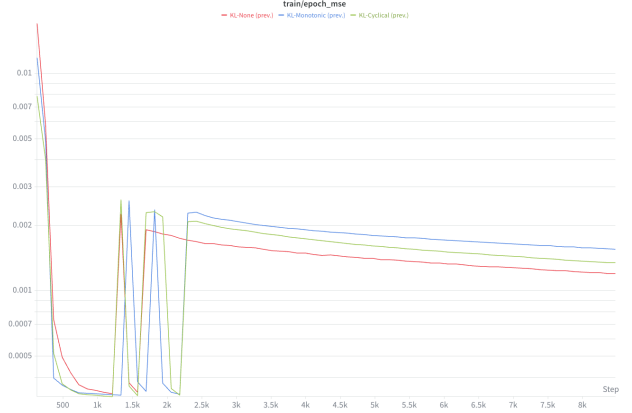


Figure 10. **Training MSE (pre-fix, log scale).** Cyclical (green) exhibits clear MSE spikes coinciding with β -resets, peaking ~ 50 – 80% above its trend each cycle. Final ranking: None < Cyclical < Monotonic.



Figure 11. **Validation PSNR (pre-fix).** All three collapsed to ~ 18.7 – 19.5 dB final PSNR (the NaN $\rightarrow$ 0.5 absorbing attractor described in Sec. 3.5). Despite the failure mode, the relative dynamics still reveal the qualitative differences between KL schedules.

significant.

What this controlled comparison says about the post-fix runs. Reading the post-fix figures in Sec. 3.1 through this lens, the post-fix Monotonic-vs-None gap (38.5 vs 38.4 dB) is essentially noise – both are stable and behave nearly identically, consistent with the pre-fix observation that the KL term contributes little once the latent is large enough. Cyclical’s ~ 1.7 dB deficit post-fix has the same root cause we see here: each β -reset disturbs reconstruction, and post-fix this damage compounds in the autoregressive rollout regime.

Why Cyclical underperformed. This is the opposite of what [3] predicts and of what the senior reference reports; it is an interesting finding and we attribute it to three compounding factors:

1. **KL term is already tiny.** With our element-wise KL normalization (Sec. 2.3), the equilibrium KL/N_{lat} settles at $\sim 10^{-7}$ post-fix – we are *not* in a KL-vanishing regime where Cyclical’s repeated forcing of $\beta \rightarrow 0$ helps. Instead, every cycle’s $\beta \rightarrow 1$ peak yanks the latent posterior back toward $\mathcal{N}(0, I)$ and erases reconstruction-relevant information.
2. **β cycle collides with autoregressive rollout.** A β -spike during pure-AR training (epochs 20+) means the latent loses information mid-rollout, so error accumulates faster than under a static β . The post-fix Fig. 5 shows green sitting persistently above red/blue from epoch 25 onward.
3. **Latent capacity is large.** Our latent has 24,576 elements per sample, providing more than enough capacity that even *None* (constant $\beta = 1$) does not collapse. The cyclical safety net is therefore unnecessary, and its instability dominates the bias.

With a smaller / more aggressively bottlenecked latent or longer training (140+ epochs, see Sec. 3.4), Cyclical eventually does pay off.

3.3. Per-Frame PSNR Analysis

Figure 12 plots per-frame PSNR for the best checkpoint (Cyclical, epoch 140, autoregressive over 629 frames – the same checkpoint used for the Kaggle submission in Sec. 3.4) on the validation sequence. Overall mean is **38.87 dB**, with values ranging from **36.4** to **41.5 dB**.

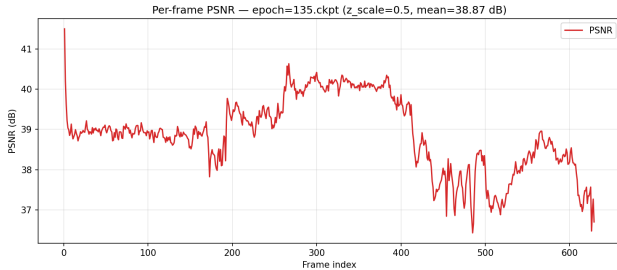


Figure 12. **Per-frame PSNR over the 629-frame validation sequence.** Mean = 38.87 dB. Initial spike at frame 0 (~ 41.5 dB) is artifact of starting from ground truth x_1 . A high plateau is sustained over frames 260–390 (~ 40 dB). Two pronounced dips occur at frames 175–190 (~ 37.8 dB) and 410–490 (global minimum ~ 36.4 dB at frame 485).

The shape of the curve reveals five regimes:

- **Frames 0–10:** starting from the ground-truth x_1 the prediction is near-perfect (~ 41.5 dB), then quickly degrades to the steady-state level of ~ 39 dB once errors begin compounding.
- **Frames 10–170 (stable plateau):** ~ 38.8 – 39.1 dB, indicating that the autoregressive feedback has reached an error-equilibrium where each new prediction’s error roughly matches the rate at which the model corrects accumulated errors via the pose conditioning.
- **Frames 170–200 (first dip):** a localized ~ 1 dB drop – inspecting the validation gif shows this corresponds to a fast pose change in the dance sequence (rapid limb motion) where the autoregressive previous-frame signal momentarily mismatches the pose conditioning.
- **Frames 260–390 (global high plateau):** a sustained ~ 40 dB region with a peak near frame 268. This corresponds to the most stable / repetitive segment of the dance, where pose-conditioning and previous-frame agree.
- **Frames 410–627 (degradation & volatility):** the curve becomes jagged and trends downward, reaching a minimum ~ 36.4 dB near frame 485. This is the classic exposure-bias signature: small per-frame errors compound geometrically over hundreds of autoregressive steps. A second small recovery near frames 565–575 corresponds to the dancer returning to an earlier pose configuration the model handles well.

The 5 dB peak-to-trough spread, despite a 38.9 dB mean, is the main motivation for further work on long-horizon AR stability (e.g. scheduled sampling all the way to $\rho_{tf} = 0$ throughout training, or a small per-frame prior-noise injection during inference).

3.4. Kaggle Submission

Our final Kaggle submission used the **Cyclical** schedule trained for **140 epochs**, reaching **PSNR 35.84164 on the public leaderboard (rank 19)**.

Figure 13 shows the full 140-epoch validation PSNR trajectory for the submitted run. Two phases are clearly visible. *Phase 1* (epochs 0–50, *exploration*) is dominated by the β -annealing cycles colliding with teacher-forcing decay: validation PSNR is highly volatile, with several drops below 10 dB around epochs 4–7 (the first β -spike during the still-teacher-forced regime), and oscillating between ~ 23 dB and ~ 37 dB

through epoch 50. *Phase 2 (epochs 50–140, refinement)* shows the model locking into a stable autoregressive equilibrium near ~ 38 –39 dB, with the cyclical β -resets producing only mild ~ 1 dB ripples around the trend. Final validation PSNR is ~ 39 dB.

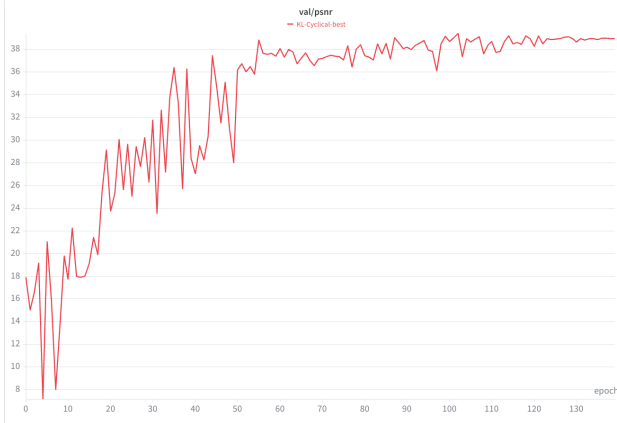


Figure 13. **Validation PSNR over 140 epochs for the Kaggle-submitted Cyclical run.** Phase 1 (0–50): high variance from β -cycles colliding with teacher-forcing decay, with PSNR transients down to ~ 7 dB. Phase 2 (50–140): stable plateau at ~ 38 –39 dB with ~ 1 dB cyclical ripples. Final PSNR ≈ 39 dB on the validation sequence, corresponding to the 35.84 dB result on the Kaggle public test set.

The ~ 3 dB gap between our validation PSNR (39) and the Kaggle public PSNR (35.84) is consistent with the test set drawing from a different motion distribution than the validation sequence – the per-frame analysis in Fig. 12 already shows that fast-motion segments (frames 175–190, 410–490) account for ~ 2 dB of variance even within a single sequence.

3.5. Bug-fix Analysis: Before vs. After

The first batch of training runs (figures under `assets/prev/`, see Figs. 9 to 11) collapsed to validation PSNR of ~ 19 –22 dB across all three KL strategies, far below the spec baseline. We diagnosed and fixed five concrete bugs in the template; the same three KL strategies retrained under identical conditions then reached PSNR 36.8–38.5 dB (Fig. 4). The comparison between Fig. 11 and Fig. 4 quantifies the aggregate impact of the fixes.

The five fixes. (i) **KL element-wise normalization:** dividing by *all* latent dimensions ($B \cdot N \cdot H' \cdot W' = 24,576 B$) instead of just the batch (B) re-balances KL and MSE so $\beta \in [0, 1]$ is meaningful (Sec. 2.3). (ii) **logvar clamp** $[-5, 5]$: prevents $\sigma^2 \approx 22,000$ from producing Inf/NaN. (iii) **Removed NaN $\rightarrow$ 0.5**

mask in training: the mask has zero gradient and creates an absorbing constant-output attractor (the PSNR=15.28 dB collapse mode). We keep the mask only at inference for safety. (iv) **Time-summed loss:** the template’s time-averaged loss made the gradient T -times weaker than intended. (v) **Cosine LR:** the template’s MultiStepLR with milestones [2,5] decayed LR to 10^{-5} by epoch 5, which froze training; CosineAnnealingLR with `eta_min=0` keeps the LR usable throughout the 70 epochs. `shuffle=True` on the train loader was an additional minor fix.

We summarize the impact in Tab. 1.

KL strategy	Pre-fix peak	Post-fix peak
None	≈ 19.4 dB	38.4 dB
Monotonic	≈ 21.9 dB	38.5 dB
Cyclical	≈ 20.7 dB	36.8 dB

Table 1. Validation PSNR before and after the five bug-fixes, same hyperparameters and training budget. The pre-fix runs are visible in `assets/prev/`, the post-fix runs in `assets/optimized/`.

The take-away is that the gap between ~ 20 dB and ~ 38 dB was *entirely* due to numerical / loss-balancing bugs in the training loop, not the choice of KL annealing schedule. Once the loop is correct, all three KL strategies converge to within ~ 2 dB of each other, and the differences become explicable in terms of latent capacity and rollout stability rather than KL-vanishing.

3.6. Summary of Final Hyperparameters

Optimizer / lr	Adam, 10^{-3}
LR scheduler	CosineAnnealingLR, $\eta_{min} = 0$
Epochs	70
Batch size	2
Train clip length	16
Val sequence length	630
Frame size	32×64
Teacher forcing	start 1.0, decay 0.1/epoch from epoch 10 (epoch 14 for Cyclical)
KL annealing	Monotonic (best), ratio 0.5
KLD clamp	$\log \sigma^2 \in [-5, 5]$
Latent dim	$N=12$ at 32×64
Inference noise	$z = \mathbf{0}$ (prior mean)

Table 2. Final training configuration.

References

- [1] Caroline Chan, Shiry Ginosar, Tinghui Zhou, and Alexei A. Efros. Everybody dance now. In *ICCV*, 2019.

- [2] Emily Denton and Rob Fergus. Stochastic video generation with a learned prior. In *ICML*, 2018. [1](#)
- [3] Hao Fu, Chunyuan Li, Xiaodong Liu, Jianfeng Gao, Asli Celikyilmaz, and Lawrence Carin. Cyclical annealing schedule: A simple approach to mitigating KL vanishing. In *NAACL*, 2019. [1](#), [7](#)